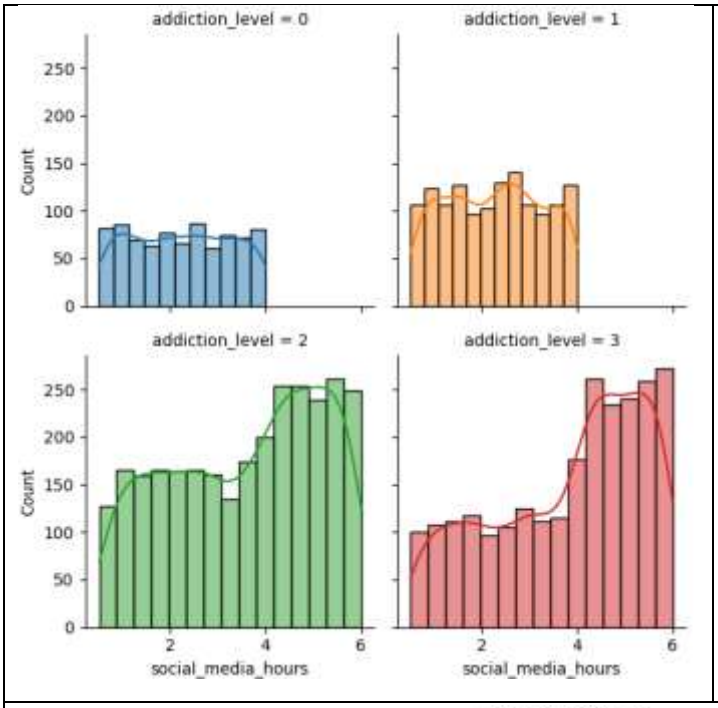
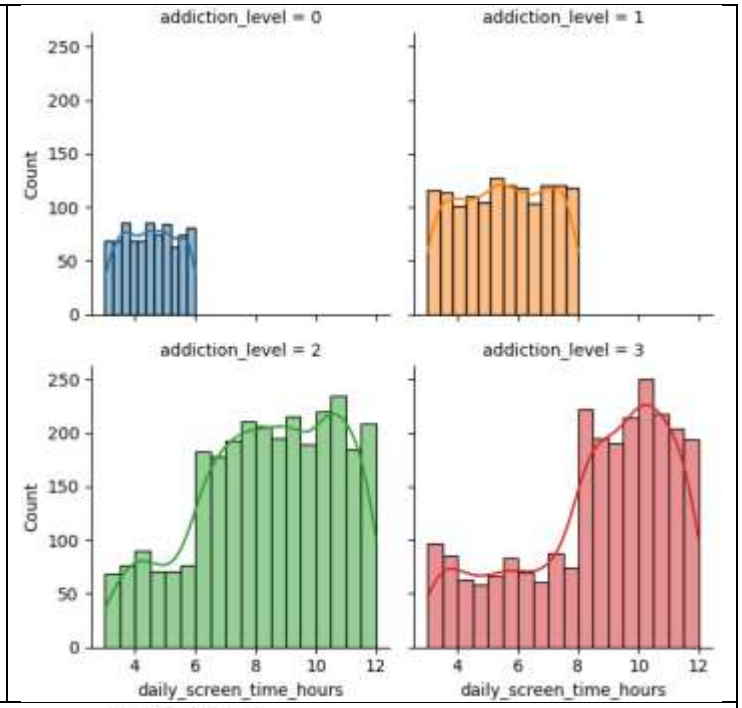
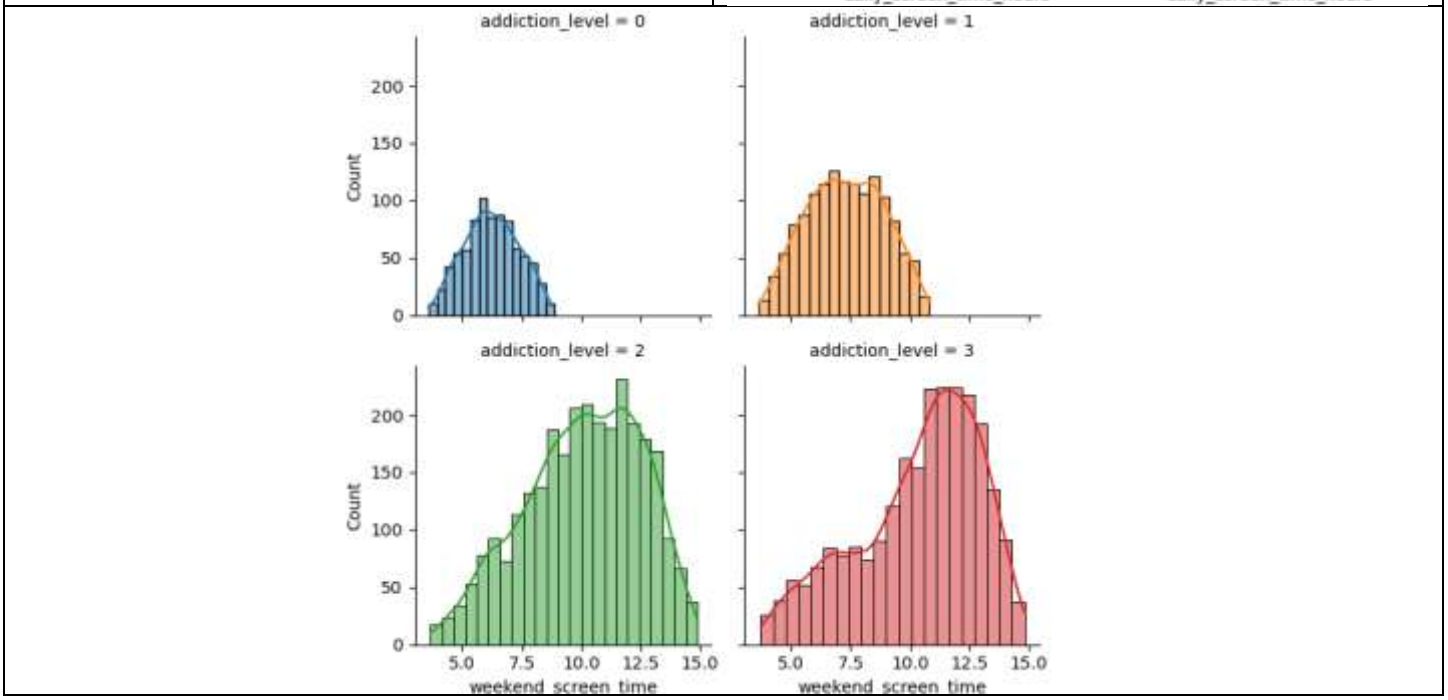

addiction_level = 0
addiction_level = 1
addiction_level = 2
addiction_level = 3
Count
social_media_hours


addiction_level = 0
addiction_level = 1
addiction_level = 2
addiction_level = 3
Count
daily_screen_time_hours


addiction_level = 0
addiction_level = 1
addiction_level = 2
addiction_level = 3
Count
weekend_screen_time

Introduction

This dataset contains over 7,500 synthetic user records designed to analyze smartphone usage patterns and predict levels of digital addiction. It simulates real-world behavioral data, including screen time, social media usage, gaming activity, sleep patterns, stress levels, and productivity impact.

The purpose of this analysis is to examine the *Smartphone Usage & Addiction Prediction* dataset in order to understand how each feature relates to the target variable, and to identify the most important features for building an effective predictive model.

To determine the best-performing model, K-Fold Cross-Validation was applied. Selected features were used to train multiple models, which were then scaled, combined using stacking techniques, and evaluated. The models were compared based on their average F1-scores, and the best model was selected accordingly.

Data cleaning and preprocessing were performed using Python. Visualizations created in both Python and Power BI illustrate how each feature relates to the target variable. The Addiction Level is categorized into four classes: No (0), Mild (1), Moderate (2), and Severe (3).Also it shows how the evaluation result is changed after stacking.

Data Exploration

- This report analyzes a dataset of 7,500+ synthetic records to predict smartphone addiction levels based on user behavior (e.g., screen time, sleep, stress, and productivity).
- The target variable, *Addiction Level*, is classified into four categories: No (0), Mild (1), Moderate (2), and Severe (3).
- Data preprocessing and visualization were performed using Python and Power BI to explore relationships between features and the target.
- Multiple models (Logistic Regression, Decision Tree, AdaBoost, Random Forest) were developed and evaluated using 5-fold cross-validation.
- A stacking approach was applied to improve performance.
- Models were compared using Accuracy, Precision, Recall, and primarily F1-score.
- Results show that stacking improves predictive performance across models.

Python Code:

1- Import libraries.

```
import numpy as np
import pandas as pd
import warnings
warnings.simplefilter(action="ignore", category=FutureWarning)
from sklearn.feature_selection import f_regression, SelectKBest
from sklearn.feature_selection import chi2
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import MinMaxScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import precision_score, recall_score, f1_score, accuracy_score
from sklearn.model_selection import KFold
from sklearn.feature_selection import RFE
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
```

2- Get Smart phone Dataset:

```
[86]:  # Read the data:
def getdata():
    PATH = ("/DataSet/")
    CSV_DATA = "Smartphone_Usage_And_Addiction_Analysis.csv"
    df  = pd.read_csv(PATH + CSV_DATA)
    print('raw data frame\n',df.head)
    return df
```

Raw date:

```
raw data frame
<bound method NDFrame.head of      transaction_id user_id  age  gender  daily_screen_time_hours  soci
al_media_hours  gaming_hours  work_study_hours  sleep_hours  notifications_per_day  app_opens_per_day
weekend_screen_time stress_level academic_work_impact addiction_level  addicted_label
0          TXN00001  U00001   21    Male                    3.23                2.01          0.89
4.55         7.55                    248                154                  3.95       Medium
Yes             NaN                 0
1          TXN00002  U00002   24   Other                    5.09                3.81          2.24
4.44         7.66                    127                 71                  6.71       Medium
Yes             NaN                 0
2          TXN00003  U00003   31   Other                    6.06                1.36          3.83
2.35         4.92                     44                106                  8.68         High
No            Mild                 0
3          TXN00004  U00004   32   Other                    7.83                5.85          1.51
3.54         8.23                    178                107                  9.77         High
Yes        Moderate                 1
4          TXN00005  U00005   25    Male                    9.96                5.92          3.42
5.27         6.21                    136                177                 12.55          Low
No          Severe                 1
5          TXN00006  U00006   26    Male                    9.32                4.26          0.29
```

3- Prepare and cleaning data:

- Replace value for target

In the data set for addiction_level values are replaced with number.

```
def replace(df):
    df['addiction_level'] = df['addiction_level'].fillna(0)
    df.loc[df['addiction_level'] == 'Severe', 'addiction_level'] = 3
    df.loc[df['addiction_level'] == 'Moderate', 'addiction_level'] = 2
    df.loc[df['addiction_level'] == 'Mild', 'addiction_level'] = 1
    return df
```

	academic_work_impact	addiction_level	addicted_label
0	Yes	0	0
1	Yes	0	0
2	No	1	0
3	Yes	2	1
4	No	3	1

- Dataset after binning:

Some binning variable were created to replace numbers instead of string. The gender, stress_level, academic_work_impact are the features that their value is changed during binning.

```
[102]: # Binning data to replace string with number
       def binning_dummy(df):
           df = pd.get_dummies(df,
                               columns=['gender','stress_level','academic_work_impact'],
                               dtype=int)
           return df
```

gender_Female	gender_Male	gender_Other	stress_level_High	stress_level_Low	stress_level_Medium	academic_work_impact_No	academic_work_impact_Yes
0	1	0	0	0	1	0	1
0	0	1	0	0	1	0	1
0	0	1	1	0	0	1	0
0	0	1	1	0	0	0	1
0	1	0	0	1	0	1	0

- Removing unnecessary features:

The addiction_lablel and user_id are unique per row and they don't contain meaningful patterns about addiction. The cause data leakage and Overfitting.

```
[103]: # remove unimportant columns
       def new_df_removeColumns(df):
           del df['transaction_id']
           del df['user_id']
           del df['addicted_label']

           return df
```

removeColumns

	age	daily_screen_time_hours	social_media_hours	gaming_hours	work_study_hours	sleep_hours	notifications_per_day	app_opens_per_day
0	21	3.23	2.01	0.89	4.55	7.55	248	154
1	24	5.09	3.81	2.24	4.44	7.66	127	71
2	31	6.06	1.36	3.83	2.35	4.92	44	106
3	32	7.83	5.85	1.51	3.54	8.23	178	107
4	25	9.96	5.92	3.42	5.27	6.21	136	177

	weekend_screen_time	addiction_level	gender_Female	gender_Male	academic_work_impact_No	academic_work_impact_Yes
0	3.95	0	0	1	0	1
1	6.71	0	0	0	0	1
2	8.68	1	0	0	1	0
3	9.77	2	0	0	0	1
4	12.55	3	0	1		

	gender_Other	stress_level_High	stress_level_Low	stress_level_Medium
0	0	0	0	1
1	1	0	0	1
2	1	1	0	0
3	1	1	0	0
4	0	0	1	0

4- Create feature pool

The method below is used to select feature for this model:

1- recursive feature elimination
2- feature importance
3- chi square

the features are selected according to their scores or orders.

The combination of features with the higher rank is chosen and tested with multiple ML models. Combination of features with best F1 score is considered for best model. Addiction_lable could be a second target and it should be ignored due to preventing data leakage. The daily_screen_time_hours , weekend_screen_time , and social_media_hours are examined to find best model.

********** RFE**********	********** Chi Feature **********	********** Importance Feature *********
age	Feature Score	feature importance
daily_screen_time_hours	9 addicted_label 2192.000000	9 addicted_label 0.187449
social_media_hours	1 daily_screen_time_hours 434.907425	1 daily_screen_time_hours 0.152824
gaming_hours	8 weekend_screen_time 277.661297	2 social_media_hours 0.124732
work_study_hours	2 social_media_hours 217.536402	8 weekend_screen_time 0.100439
sleep_hours		4 work_study_hours 0.066686
notifications_per_day		3 gaming_hours 0.066087
app_opens_per_day		5 sleep_hours 0.065973
weekend_screen_time		6 notifications_per_day 0.064552
addicted_label		7 app_opens_per_day 0.063608
gender_Female		0 age 0.045559
gender_Male		14 stress_level_Low 0.008431
gender_Other		15 stress_level_Medium 0.007978
stress_level_High		13 stress_level_High 0.007732
stress_level_Low		11 gender_Male 0.007691
stress_level_Medium		12 gender_Other 0.007651
academic_work_impact_No		16 academic_work_impact_No 0.007644
academic_work_impact_Yes		10 gender_Female 0.007623
		17 academic_work_impact_Yes 0.007339

```python
def feature_importance_randomForest(df,target):
    # Labels are the values we want to predict
    labels = np.array(df[target])

    # Remove the labels from the features
    # axis 1 refers to the columns
    features = df.drop(target, axis=1)

    # Saving feature names for later use
    feature_list = list(features.columns)

    # Convert to numpy array
    features = np.array(features)

    # Split the data into training and testing sets
    train_features, test_features, train_labels, test_labels = \
        train_test_split(features, labels, test_size=0.15, random_state=42)

    # Instantiate model with 1000 decision trees
    rf = RandomForestClassifier(n_estimators=100)

    # Train the model on training data
    rf.fit(train_features, train_labels)

    # Use the forest's predict method on the test data
    predictions = rf.predict(test_features)

    # Get numerical feature importances
    importances = list(rf.feature_importances_)

    dfImportances = pd.DataFrame()
    for i in range(0, len(feature_list)):
        dfImportances = \
            dfImportances._append({"feature": feature_list[i], "importance": importances[i]},
                                  ignore_index=True)
    dfImportances = dfImportances.sort_values(by=["importance"], ascending=False)
    return dfImportances
```

```python
# Chi square to determine best features
def chi(df, target):
    _list = [[],[]]

    X = df.copy()
    del X[target]
    y = fit_transform_target(df, target)

    # Not allowed to have negative X values in KBest so scale with MinMax.
    scalerX = MinMaxScaler()
    scaledX = scalerX.fit_transform(X)

    selector = SelectKBest(score_func=chi2, k='all')
    selector.fit(scaledX, y)

    # Create DataFrame directly from the original attributes
    df_chi = pd.DataFrame({
        'Feature': X.columns,
        'Score': selector.scores_
    })

    # Filter and sort by score in descending order
    df_sorted = df_chi[df_chi['Score'] > 3.8].sort_values(by='Score', ascending=False)

    return df_sorted
```

```python
def REF_featureSelection(df,target):
    # Seperate the target and independent variable
    X = df.copy()  # Create separate copy to prevent unwanted tampering of data.
    del X[target]  # Delete target variable.

    # Target variable
    y = df[target]

    # Create the object of the model
    model = LogisticRegression(solver='saga', max_iter=10000)

    # Specify the number of  features to select
    rfe = RFE(model, n_features_to_select=30)

    # fit the model
    rfe = rfe.fit(X, y)

    RFE_result = [list(X.keys())[i] for i in range(len(X.keys())) if rfe.support_[i]]
    # Show top features.
    for i in range(0, len(X.keys())):
        if (rfe.support_[i]):
            print(X.keys()[i])

    return RFE_result
```

5- Choose best feature

After selecting features, they are split and stacked and scaled to find the best result by adding or removing other features to the data frame. The code was run with K-fold and we have multiple F1 score for each model across all fold. The F1 average is used to select best model.

According to the F1 Average result, second row has better results than the first one.

Features with the best F1 score are:

- daily_screen_time_hours
- weekend_screen_time
- social_media_hours

F1 average score -table.

Features	DecisionTreeClassifier	AdaBoostClassifier	RandomForestClassifier	LogisticRegression
daily_screen_time_hours, social_media_hours	0.4820	0.362	0.472	0.42
daily_screen_time_hours, social_media_hours, weekend_screen_time	0.512	0.372	0.531	0.434

6- Train, stack, and scale models. Select best model

Here is the python code to train, fit, stack and scale dataset. F1 average is calculated and best model is selected.

To fit the base models the user defined fitBaseModels() function is using 3 models. The tree models are: DecisionTreeClassifire(), RandomForestClassifire(), and LogisticRegression().

```
def getUnfitModels():
    models = list()
    models.append(LogisticRegression(max_iter=10000))
    models.append(DecisionTreeClassifier())
    models.append(RandomForestClassifier(n_estimators=100))
    return models
```

```
def fitBaseModels(X_train, y_train, X_test, models):
    dfPredictions = pd.DataFrame()

    # Fit base model and store its predictions in dataframe.
    for i in range(0, len(models)):
        models[i].fit(X_train, y_train)
        predictions = models[i].predict(X_test)
        colName = str(i)
        dfPredictions[colName] = predictions
    return dfPredictions, models
```

```
    # Fit base models.
    dfPredictions, models = fitBaseModels(X_train_kf, y_train_kf, X_val_kf, unfitModels)
    dfPredictions_List.append(dfPredictions)
```

```python
# Split data into train, test and validation sets.
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.80)
X_test, X_val, y_test, y_val = train_test_split(X_temp, y_temp, test_size=0.50)


for train_index, val_index in kfold.split(X_test):

    dfValidationPredictions = pd.DataFrame()

    X_train_kf, X_val_kf = X_test.iloc[train_index], X_test.iloc[val_index]
    y_train_kf, y_val_kf = y_test.iloc[train_index], y_test.iloc[val_index]

    # create a lost of y_val_kf to use fit best stacked model
    y_val_kfList.append(y_val_kf)

    # Get base models.
    unfitModels = getUnfitModels()

    # Fit base models.
    dfPredictions, models = fitBaseModels(X_train_kf, y_train_kf, X_val_kf, unfitModels)
    dfPredictions_List.append(dfPredictions)

    count += 1
    # print('\n','Kfold #', count)
    for i in range(0, len(models)):
        predictions = models[i].predict(X_val_kf)
        colName = str(i)
        dfValidationPredictions[colName] = predictions
        # print("** Evaluate Base Models **")
        precision, recall, f1, accuracy = evaluateModel(y_val_kf, predictions, models[i])

        model = models[i]
        if models[i] == 'LogisticRegression':
            model = LogisticRegression(multi_class='multinomial', fit_intercept=True, solver='lbfgs',
                                       max_iter=10000)
        elif models[i] == 'DecisionTreeClassifier':
            model = DecisionTreeClassifier(max_depth=3)
        elif models[i] == 'RandomForestClassifier':
            model = RandomForestClassifier(n_estimators=100)
# Stack each base model:
stackedModel = fitStackedModel(dfPredictions, y_val_kf, model)
# Evaluate stacked model with validation data.
stackedPredictions = stackedModel.predict(dfPredictions)
# print("** Evaluate Stacked Model **")
precision_stk, recall_stk, f1_stk, accuracy_stk = evaluateModel(y_val_kf, stackedPredictions, stackedMod
```

```
if 'LogisticRegression' in str(models[i]):
    print('LogisticRegression_coef\n',model.coef_,'\n',X.columns)
else:
    print(str(models[i]),'\n',model.feature_importances_,'\n',X.columns)

if 'LogisticRegression' in str(models[i]):

    f1List_LogisticRegression.append(f1)
    recallList_LogisticRegression.append(recall)
    accuracyList_LogisticRegression.append(accuracy)
    precisionList_LogisticRegression.append(precision)

    f1List_LogisticRegression_stk.append(f1_stk)
    recallList_LogisticRegression_stk.append(recall_stk)
    accuracyList_LogisticRegression_stk.append(accuracy_stk)
    precisionList_LogisticRegression_stk.append(precision_stk)

elif 'DecisionTreeClassifier' in str(models[i]):

    f1List_DecisionTreeClassifier.append(f1)
    recallList_DecisionTreeClassifier.append(recall)
    accuracyList_DecisionTreeClassifier.append(accuracy)
    precisionList_DecisionTreeClassifier.append(precision)

    f1List_DecisionTreeClassifier_stk.append(f1_stk)
    recallList_DecisionTreeClassifier_stk.append(recall_stk)
    accuracyList_DecisionTreeClassifier_stk.append(accuracy_stk)
    precisionList_DecisionTreeClassifier_stk.append(precision_stk)

elif 'RandomForestClassifier' in str(models[i]):

    f1List_RandomForestClassifier.append(f1)
    recallList_RandomForestClassifier.append(recall)
    accuracyList_RandomForestClassifier.append(accuracy)
    precisionList_RandomForestClassifier.append(precision)

    f1List_RandomForestClassifier_stk.append(f1_stk)
    recallList_RandomForestClassifier_stk.append(recall_stk)
    accuracyList_RandomForestClassifier_stk.append(accuracy_stk)
    precisionList_RandomForestClassifier_stk.append(precision_stk)
```

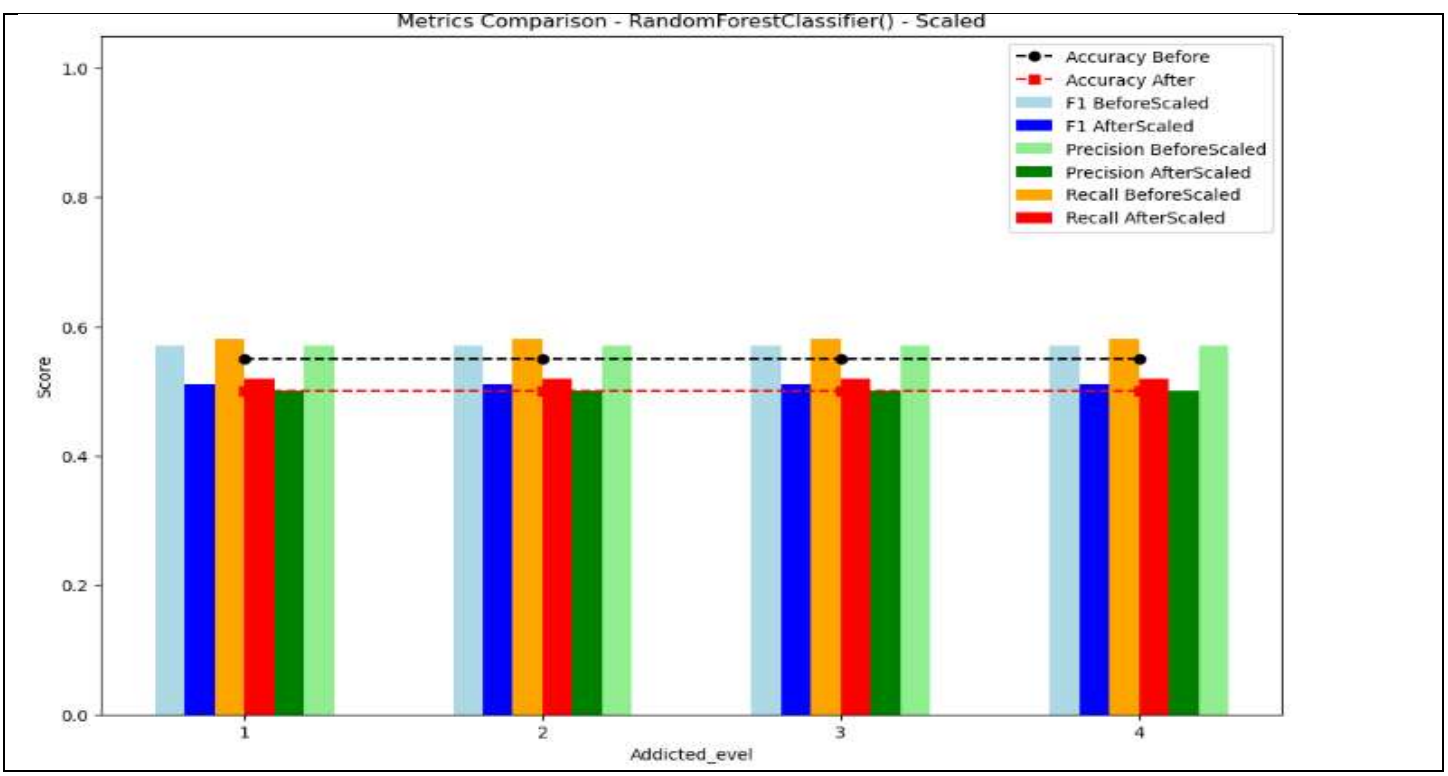
```
# find best model among all basemodel via F1_mean across all kfold
f1_mean_LogisticRegression = np.mean(f1List_LogisticRegression)
f1_mean_DecisionTreeClassifier = np.mean(f1List_DecisionTreeClassifier)
f1_mean_RandomForestClassifier = np.mean(f1List_RandomForestClassifier)
f1_mean = {'f1_mean_LogisticRegression':f1_mean_LogisticRegression,
           'f1_mean_DecisionTreeClassifier': f1_mean_DecisionTreeClassifier,
           'f1_mean_RandomForestClassifier': f1_mean_RandomForestClassifier}
print( "f1 Average for all models:   " +
       " \nDecisionTreeClassifier:" + str(f1_mean_DecisionTreeClassifier) +
       " \nRandomForestClassifier:" + str(f1_mean_RandomForestClassifier) +
       " \nLogisticRegression:" + str(f1_mean_LogisticRegression) \
       )

max_f1 = max(f1_mean, key=f1_mean.get)
bestmodel = max_f1.split('_')[2]
```

Here is the calculated F1 Average:

```
f1 Average for all models:
DecisionTreeClassifier:0.512
AdaBoostClassifier:0.372
RandomForestClassifier:0.5319999999999999
LogisticRegression:0.434
** Best Model Across folds is ** RandomForestClassifier
** Evaluation of best scaled:  RandomForestClassifier  model ***
RandomForestClassifier
Precision:0.57
Recall:0.58
F1:0.57
Accuracy:0.55
```

The tree based models does not need to be scaled due to decreasing the scores after scaling, but if the best model is logesticregression, it should be scaled.

This diagram shows how scaling decreased score for a Tree based model:



7- Comparison diagram to compare before and after stack.

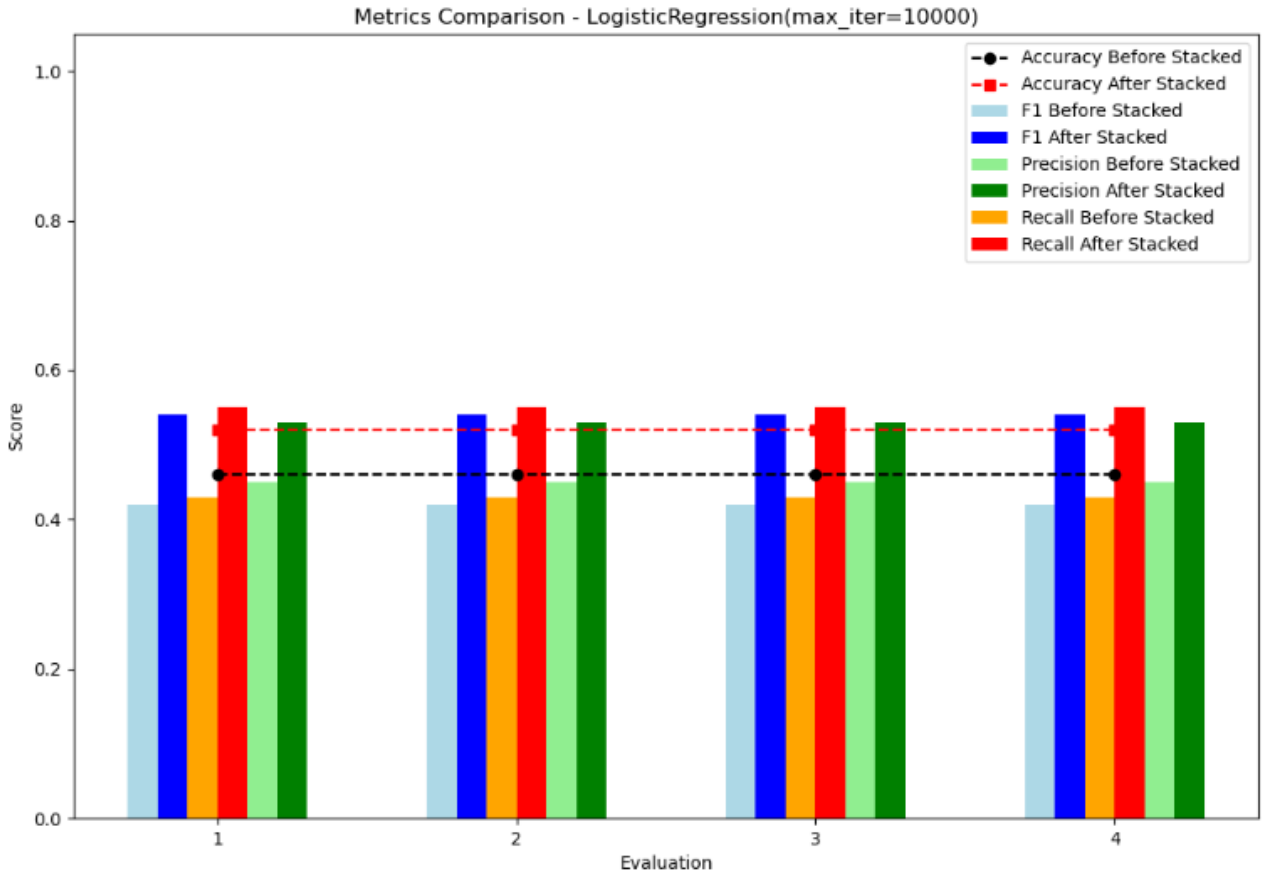
Stacking the model can improve it and the result after stacking the scores are increased significantly.

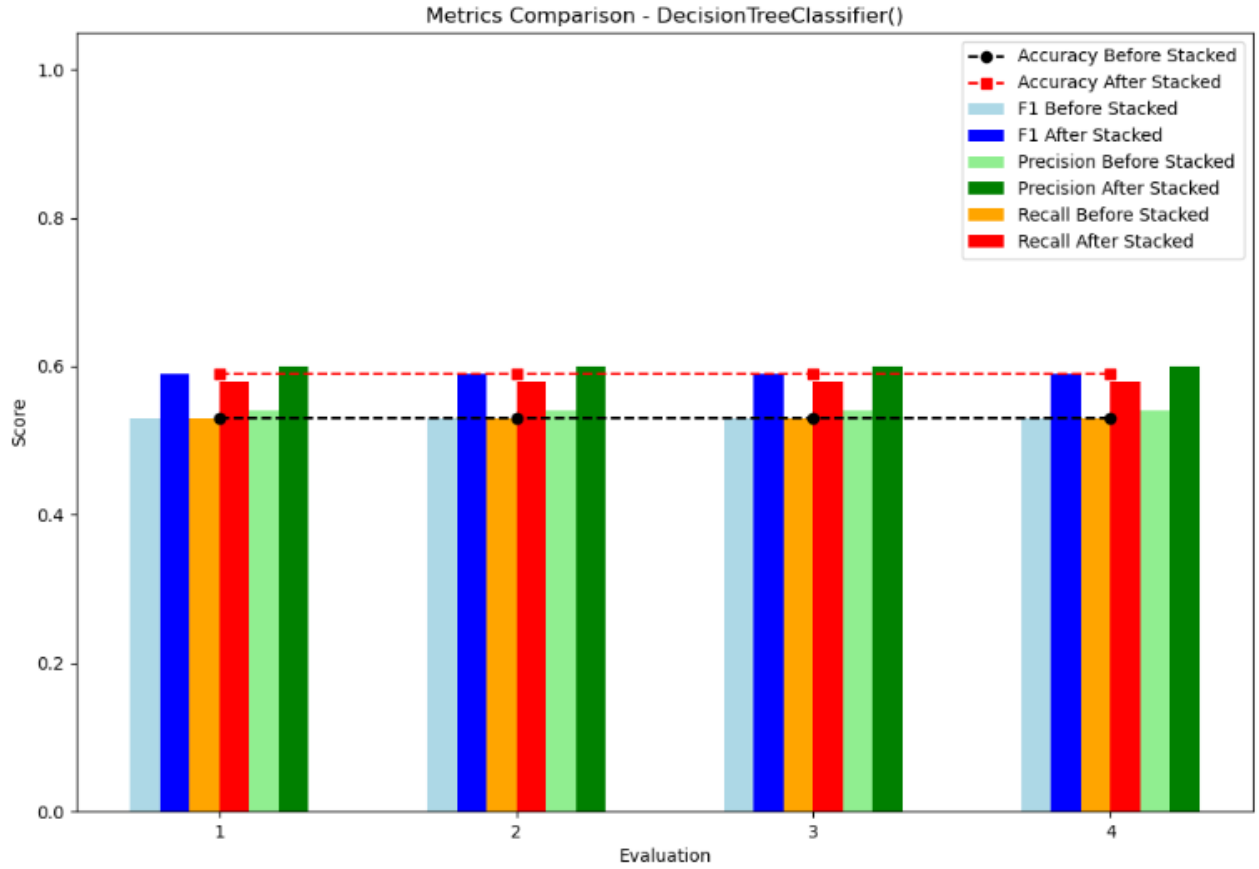
```
# Stack each base model:
stackedModel = fitStackedModel(dfPredictions, y_val_kf, model)
# Evaluate stacked model with validation data.
stackedPredictions = stackedModel.predict(dfPredictions)
# print("** Evaluate Stacked Model **")
precision_stk, recall_stk, f1_stk, accuracy_stk = evaluateModel(y_val_kf, stackedPredictions, stackedModel)
```

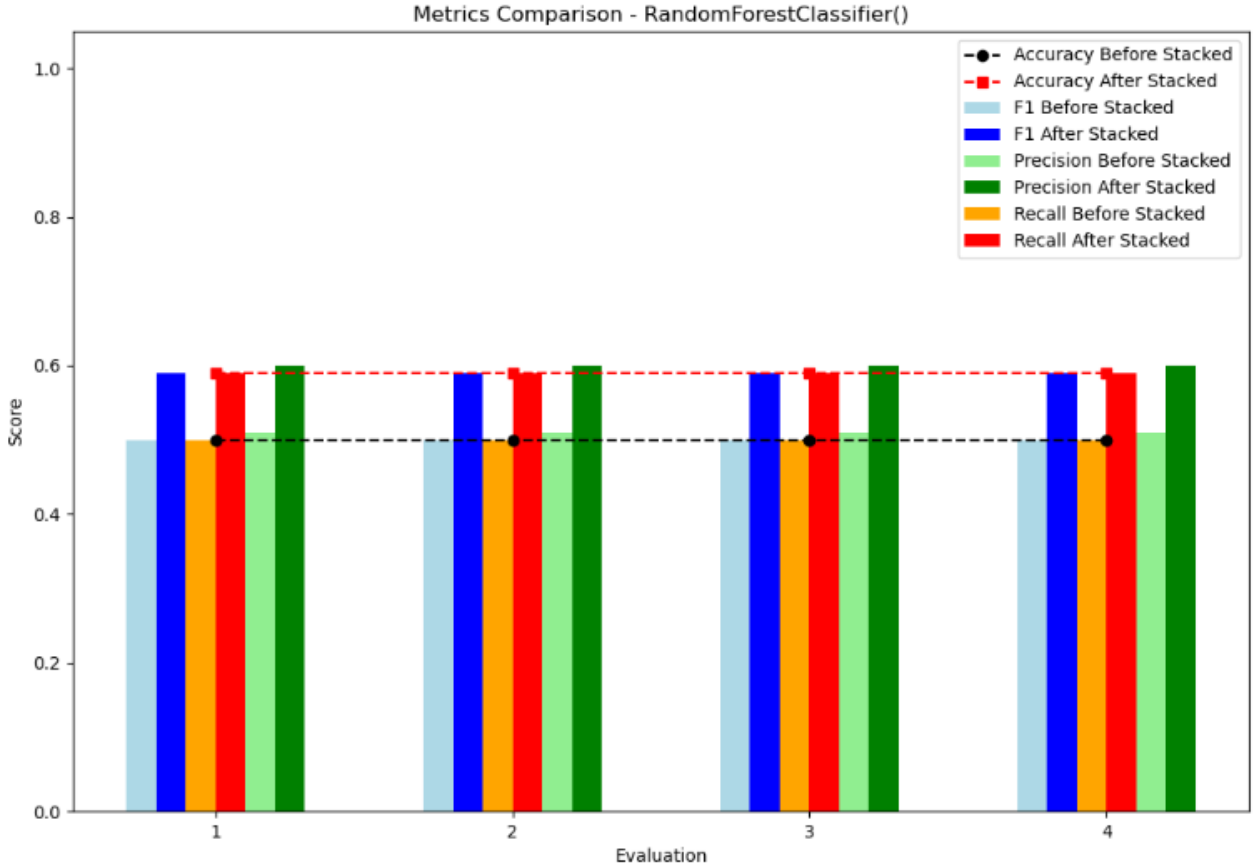
This is the code to create diagram to compare scores before and after stacke

```
# bar chart used to compare the evaluation result after scale
classes = ['1', '2', '3', '4']
metrics = ['F1', 'Precision', 'Recall', 'Accuracy']  # Accuracy repeated per class

def plot_model_comparison(model_name, Diagram_status, f1_before, f1_after,
                          precision_before, precision_after,
                          recall_before, recall_after,
                          accuracy_before, accuracy_after):
    x = np.arange(len(classes))
    width = 0.10  # smaller width to fit multiple bars

    plt.figure(figsize=(12, 8))

    # Plot F1
    plt.bar(x - width * 2.5, f1_before, width, label='F1 Before'+Diagram_status, color='lightblue
    plt.bar(x - width * 1.5, f1_after, width, label='F1 After'+Diagram_status, color='blue')

    # Plot Precision
    plt.bar(x + width * 2.5, precision_before, width, label='Precision Before'+Diagram_status, co
    plt.bar(x + width * 1.5, precision_after, width, label='Precision After'+Diagram_status, colo

    # Plot Recall
    plt.bar(x - width * 0.5, recall_before, width, label='Recall Before'+Diagram_status, color='o
    plt.bar(x + width * 0.5, recall_after, width, label='Recall After'+Diagram_status, color='red

    acc_before = [accuracy_before] * len(classes)
    acc_after = [accuracy_after] * len(classes)
    plt.plot(x, acc_before, marker='o', linestyle='--', color='black', label='Accuracy Before')
    plt.plot(x, acc_after, marker='s', linestyle='--', color='red', label='Accuracy After')

    plt.xticks(x, classes)
    plt.xlabel('Addicted_evel')
    plt.ylabel('Score')
    plt.ylim(0, 1.05)
    plt.title(f'Metrics Comparison - {model_name} - {Diagram_status}')
    plt.legend()
    plt.show()
    # # Diagram of best model compare scores before and after scaling
    plot_model_comparison(model, 'Scaled', f1, f1_scl,
                          precision, precision_scl,
                          recall, recall_scl,
                          accuracy, accuracy_scl)
```

Diagram shows how values are changed after stacking the models:

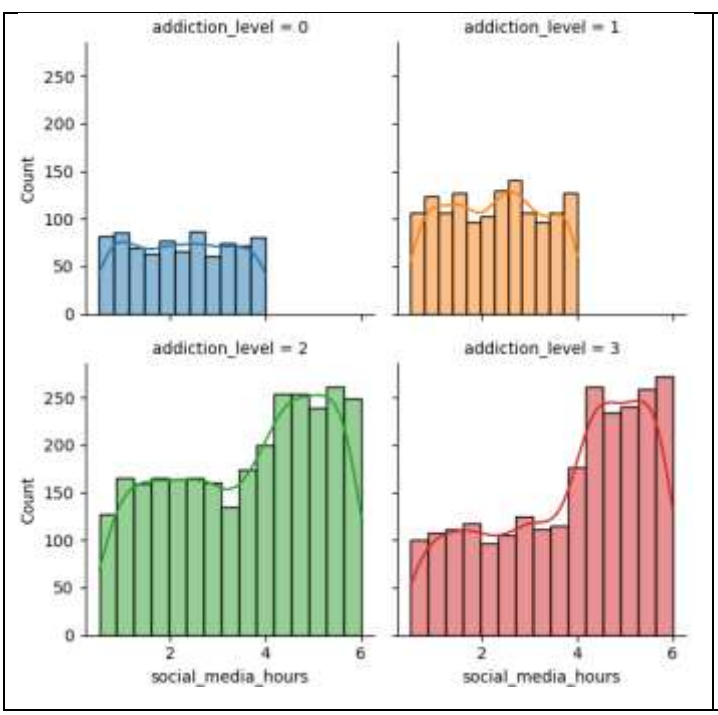


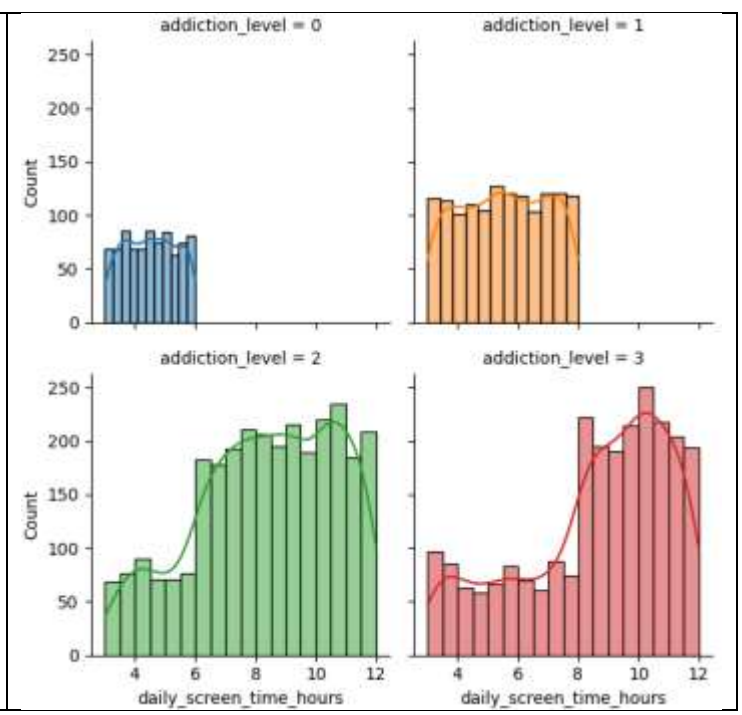


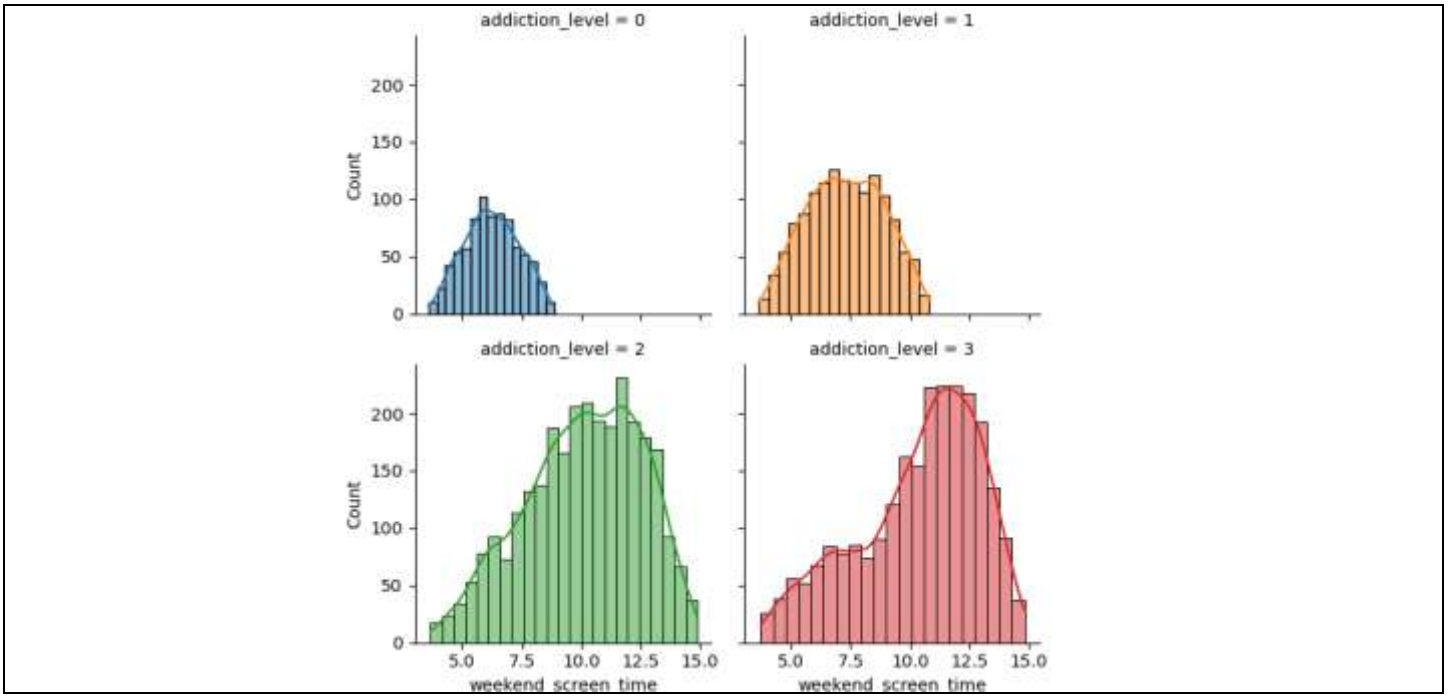


8- Correlation of all features and target via histogram in Python.

These histograms shows how **Usage Behavior features** behave across **addiction levels (0–3)** and indicate a positive relationship between daily screen time, social_media_hours, weekend_screen_time and addiction level. Higher addiction levels are associated with increased features, with level 3 showing a strong concentration in higher usage ranges. Based on the clear and distinct shift in the centers of these distributions, they have a strong relationship with the target.



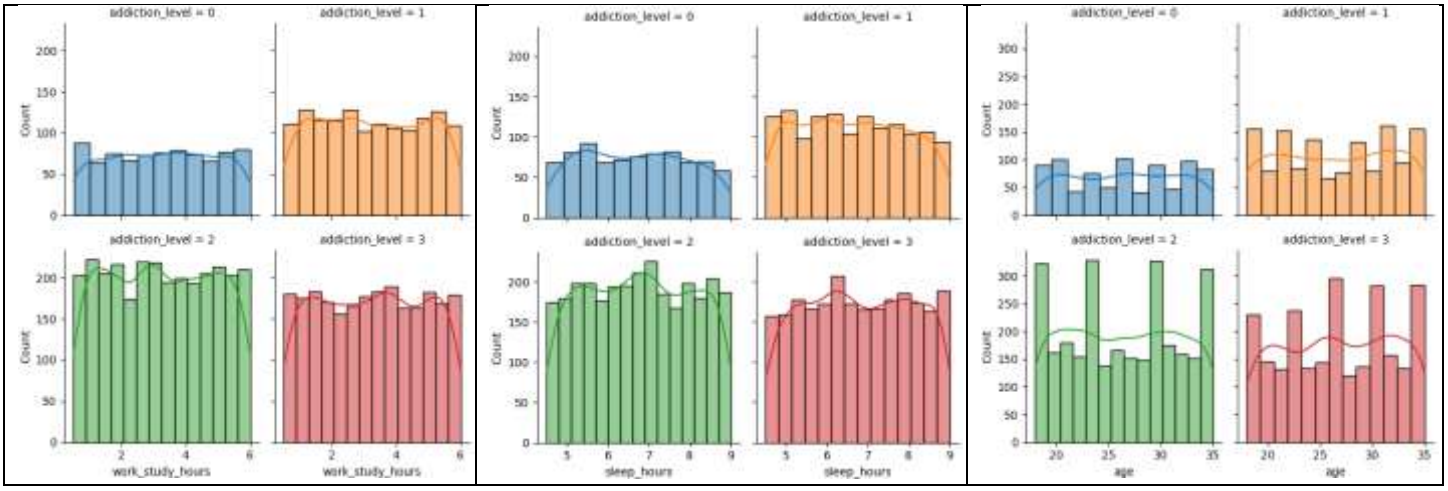


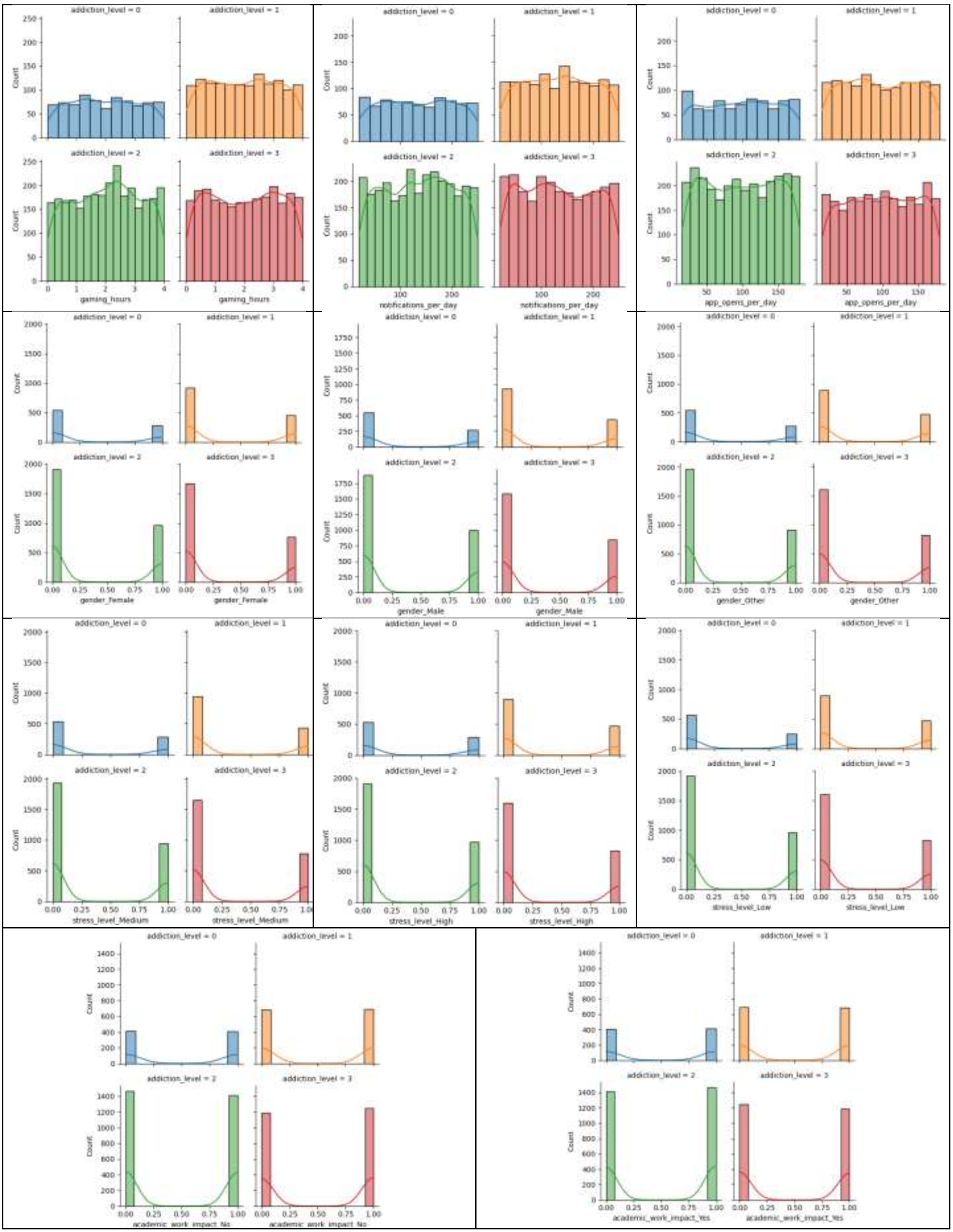


The visual analysis indicates that features within the **Engagement Intensity**, **Lifestyle & Health**, **Productivity**, and **Demographics** categories exhibit weak or negligible relationships with the target variable (*addiction_level*). Across these features, the distributions remain largely uniform for all classes, showing no meaningful shifts in central tendency or dispersion.

Specifically, variables such as *work_study_hours, sleep_hours, age, gaming_hours, notifications_per_day, app_opens_per_day, gender, stress,* and *academic_work_impact* display consistent shapes across different addiction levels. This uniformity suggests a lack of discriminative power, as the feature values do not vary systematically with changes in the target.

From a statistical perspective, this pattern is typically associated with low correlation coefficients and non-significant differences in group means. Consequently, these features are unlikely to contribute substantially to predictive performance and may have limited importance in classification models.




addiction_level = 0
addiction_level = 1
addiction_level = 2
addiction_level = 3
Count
gaming_hours
notifications_per_day
app_opens_per_day
gender_Female
gender_Male
gender_Other
stress_level_Medium
stress_level_High
stress_level_Low
academic_work_impact_No
academic_work_impact_Yes

9- Select features via diagrams

In conclusion according to the result of histograms and comparing the scores across all folds in python the only features that have the significant positive relation on the target are

- daily_screen_time_hours
- social_media_hours
- weekend_screen_time

The best features are:

```
df[[
    daily_screen_time_hours,
    social_media_hours,
    weekend_screen_time
]]
```